

即状态图并不是 DAG。如果直接用记忆化搜索，会出现无限递归。

正确的方法是把状态看成结点，状态转移看成边，转化成图论中的最短路径问题，然后使用 Dijkstra 或 Bellman-Ford 算法求解。不过这道题和普通的最短路径问题不一样：结点很多，多达 2^n 个，而且很多状态根本遇不到（即不管怎么打补丁，也不可能打成那个状态），所以没有必要像前面那样先把图储存好。

还记得第 7 章中介绍的“隐式图搜索”吗？这里也可以用相同的方法：当需要得到某个结点 u 出发的所有边时，不是去读 $G[u]$ ，而是直接枚举所有 m 个补丁，看看是否能打得上。不管是 Dijkstra 算法还是 Bellman-Ford 算法，这个方法都适用。本题很经典，强烈建议读者编程实现。

11.4 网络流初步

网络流是一个适用范围相当广的模型，相关的算法也非常多。尽管如此，网络流中的概念、思想和基本算法并不难理解。

11.4.1 最大流问题

如图 11-9 所示，假设需要把一些物品从结点 s （称为源点）运送到结点 t （称为汇点），可以从其他结点中转。图 11-9（a）中各条有向边的权表示最多能有多少个物品从这条边的起点直接运送到终点。例如，最多可以有 9 个物品从结点 v_3 运送到 v_2 。

图 11-9（b）展示了一种可能的方案，其中每条边中的第一个数字表示实际运送的物品数目，而第二个数字就是题目中的上限。

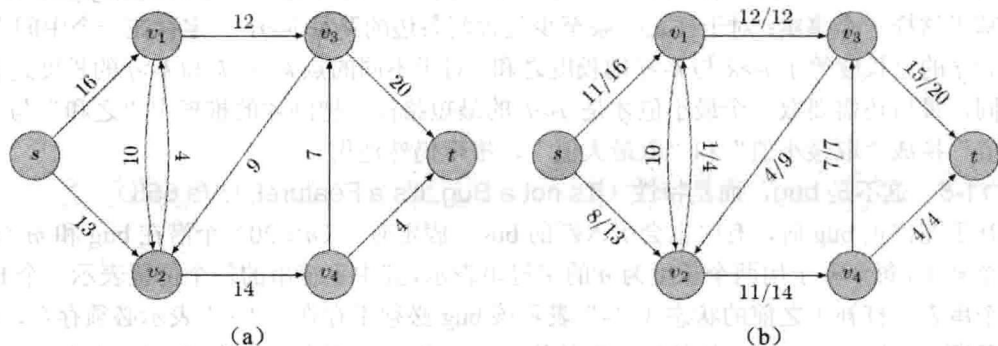


图 11-9 物资运送问题

这样的问题称为最大流问题（Maximum-Flow Problem）。对于一条边 (u, v) ，它的物品上限称为容量（capacity），记为 $c(u, v)$ （对于不存在的边 (u, v) ， $c(u, v) = 0$ ）；实际运送的物品称为流量（flow），记为 $f(u, v)$ 。注意，“把 3 个物品从 u 运送到 v ，又把 5 个物品从 v 运送到 u ”没什么意义，因为它等价于把两个物品从 v 运送到 u 。这样，就可以规定 $f(u, v)$ 和 $f(v, u)$ 最多只有一个正数（可以均为 0），并且 $f(u, v) = -f(v, u)$ 。这样规定就好比“把 3 个物

品从 u 运送到 v ”等价于“把 -3 个物品从 v 运送到 u ”一样。

最大流问题的目标是把最多的物品从 s 运送到 t ，而其他结点都只是中转，因此对于除了结点 s 和 t 外的任意结点 u ， $\sum_{(u,v) \in E} f(u,v) = 0$ （这些 f 中有些是负数）。从 s 运送出来的物品数目等于到达 t 的物品数目，而这正是此处最大化的目标。

提示 11-2：在最大流问题中，容量 c 和流量 f 满足 3 个性质：容量限制 ($f(u,v) \leq c(u,v)$)、斜对称性 ($f(u,v) = -f(v,u)$) 和流量平衡 (对于除了结点 s 和 t 外的任意结点 u ， $\sum_{(u,v) \in E} f(u,v) = 0$)。

问题的目标是最大化 $|f| = \sum_{(s,v) \in E} f(s,v) = \sum_{(u,t) \in E} f(u,t)$ ，即从 s 点流出的净流量（它也等于流入 t 点的净流量）。

11.4.2 增广路算法

介绍完最大流问题后，下面介绍求解最大流问题的算法。算法思想很简单，从零流（所有边的流量均为 0）开始不断增加流量，保持每次增加流量后都满足容量限制、斜对称性和流量平衡 3 个条件。

计算出图 11-10 (a) 中的每条边上容量与流量之差（称为残余容量，简称残量），得到图 11-10 (b) 中的残量网络 (residual network)。同理，由图 11-10 (c) 可以得到图 11-10 (d)。注意残量网络中的边数可能达到原图中边数的两倍，如原图中 $c=16, f=11$ 的边在残量网络中对应正反两条边，残量分别为 $16-11=5$ 和 $0-(-11)=11$ 。

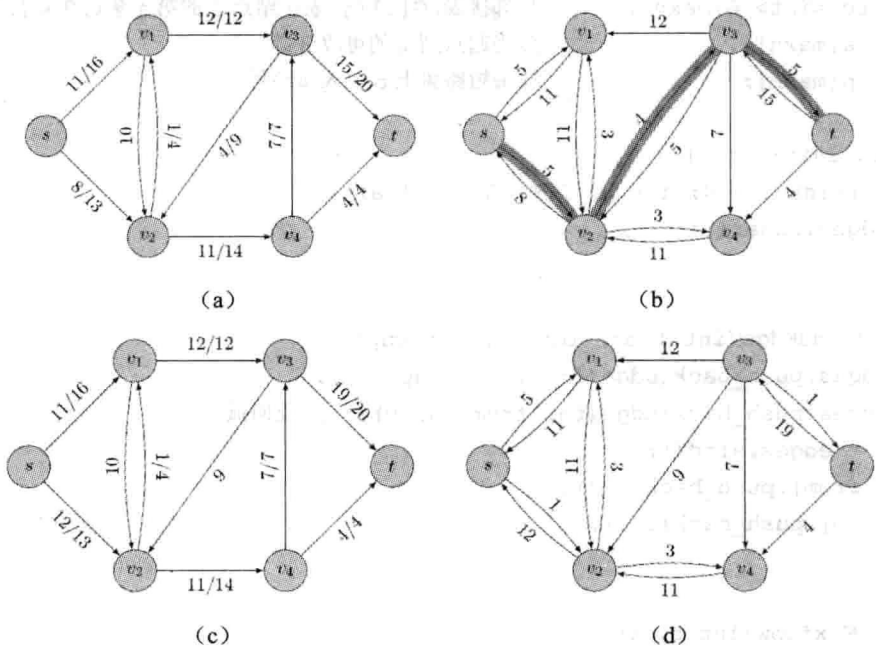


图 11-10 残量网络和增广路算法

该算法基于这样一个事实：残量网络中任何一条从 s 到 t 的有向道路都对应一条原图中

的增广路 (augmenting path) ——只要求出该道路中所有残量的最小值 d , 把对应的所有边上的流量增加 d 即可, 这个过程称为增广 (augmenting)。不难验证, 如果增广前的流量满足 3 个条件, 增广后仍然满足。显然, 只要残量网络中存在增广路, 流量就可以增大。可以证明它的逆命题也成立: 如果残量网络中不存在增广路, 则当前流就是最大流。这就是著名的增广路定理。

提示 11-3: 当且仅当残量网络中不存在 s - t 有向道路 (增广路) 时, 此时的流是从 s 到 t 的最大流。

“找任意路径”最简单的办法无疑是用 DFS, 但很容易找出让它很慢的例子。一个稍微好一些的方法是使用 BFS, 它足以应对数据不刁钻的网络流题目。这就是 Edmonds-Karp 算法。下面是完整的代码。注意 Edge 结构体多了 flow 和 cap 两个变量, 但是 AddEdge 却和 Dijkstra 中的同名函数很接近。这便是得益于 Edge 结构体这一设计。

```
struct Edge {
    int from, to, cap, flow;
    Edge(int u, int v, int c, int f):from(u),to(v),cap(c),flow(f) {}
};

struct EdmondsKarp {
    int n, m;
    vector<Edge> edges;           //边数的两倍
    vector<int> G[maxn];         //邻接表, G[i][j]表示结点 i 的第 j 条边在 e 数组中的序号
    int a[maxn];                 //当起点到 i 的可改进量
    int p[maxn];                 //最短路树上 p 的入弧编号

    void init(int n) {
        for(int i = 0; i < n; i++) G[i].clear();
        edges.clear();
    }

    void AddEdge(int from, int to, int cap) {
        edges.push_back(Edge(from, to, cap, 0));
        edges.push_back(Edge(to, from, 0, 0)); //反向弧
        m = edges.size();
        G[from].push_back(m-2);
        G[to].push_back(m-1);
    }

    int Maxflow(int s, int t) {
        int flow = 0;
        for(;;) {
            memset(a, 0, sizeof(a));
```

```

queue<int> Q;
Q.push(s);
a[s] = INF;
while(!Q.empty()) {
    int x = Q.front(); Q.pop();
    for(int i = 0; i < G[x].size(); i++) {
        Edge& e = edges[G[x][i]];
        if(!a[e.to] && e.cap > e.flow) {
            p[e.to] = G[x][i];
            a[e.to] = min(a[x], e.cap - e.flow);
            Q.push(e.to);
        }
    }
    if(a[t]) break;
}
if(!a[t]) break;
for(int u = t; u != s; u = edges[p[u]].from) {
    edges[p[u]].flow += a[t];
    edges[p[u]^1].flow -= a[t];
}
flow += a[t];
return flow;
}
};

```

注意上面代码中的一个技巧：每条弧和对应的反向弧保存在一起。边 0 和 1 互为反向边；边 2 和 3 互为反向边……一般地，边 i 的反向边为 i^1 ，其中“ $\wedge$ ”为二进制异或运算符（想一想，为什么）。

正如所见，上面的代码和普通 BFS 并没有太大的不同。唯一需要注意的是，在扩展结点的同时还需递推出从 s 到每个结点 i 的路径上的最小残量 $a[i]$ ，则 $a[t]$ 就是整条 $s-t$ 道路上的最小残量。另外，由于 $a[i]$ 总是正数，所以用它代替了原来的 vis 标志数组。上面的代码把流初始化为零流，但这并不是必需的。只要初始流是可行的（满足 3 个限制条件），就可以用增广路算法进行增广。

11.4.3 最小割最大流定理

有一个与最大流关系密切的问题：最小割。如图 11-11 所示，把所有顶点分成两个集合 S 和 $T=V-S$ ，其中源点 s 在集合 S 中，汇点 t 在集合 T 中。

如果把“起点在 S 中，终点在 T 中”的边全部删除，就无法从 s 到达 t 了。这样的集合划分 (S, T) 称为一个 $s-t$ 割，它的容量定义为： $c(S, T) = \sum_{u \in S, v \in T} c(u, v)$ ，即起点在 S 中，终点在 T 中的所有边的容量和。

还可从另外一个角度看待割。如图 11-12 所示, 从 s 运送到 t 的物品必然通过跨越 S 和 T 的边, 所以从 s 到 t 的净流量等于 $|f| = f(S, T) = \sum_{u \in S, v \in T} f(u, v) \leq \sum_{u \in S, v \in T} c(u, v) = c(S, T)$ 。

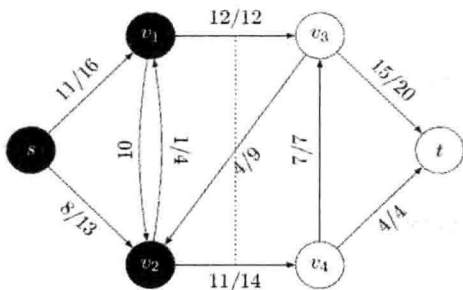


图 11-11 网络中的割

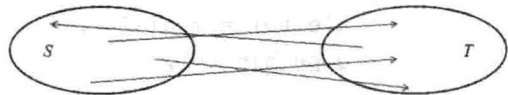


图 11-12 流和割的关系

注意这里的割 (S, T) 是任取的, 因此得到了一个重要结论: 对于任意 $s-t$ 流 f 和任意 $s-t$ 割 (S, T) , 有 $|f| \leq c(S, T)$ 。

下面来看残量网络中没有增广路的情形。既然不存在增广路, 在残量网络中 s 和 t 并不连通。当 BFS 没有找到任何 $s-t$ 道路时, 把已标号结点 ($a[u] > 0$ 的结点 u) 集合看成 S , 令 $T = V - S$, 则在残量网络中 S 和 T 分离, 因此在原图中跨越 S 和 T 的所有弧均满载 (这样的边才不会存在于残量网络中), 且没有从 T 回到 S 的流量, 因此 $|f| \leq c(S, T)$ 成立。

前面说过, 对于任意的 f 和 (S, T) , 都有 $|f| \leq c(S, T)$, 而此处又找到了一组让等号成立的 f 和 (S, T) 。这样, 便同时证明了增广路定理和最小割最大流定理: 在增广路算法结束时, f 是 $s-t$ 最大流, (S, T) 是 $s-t$ 最小割。

提示 11-4: 增广路算法结束时, 令已标号结点 ($a[u] > 0$ 的结点) 集合为 S , 其他结点集合为 $T = V - S$, 则 (S, T) 是图的 $s-t$ 最小割。

11.4.4 最小费用最大流问题

下面给网络流增加一个因素: 费用。假设每条边除了有一个容量限制外, 还有一个单位流量所需的费用 (cost)。图 11-13 (a) 中分别用 c 和 a 来表示每条边的容量和费用, 而图 11-13 (b) 给出了一个在总流量最大的前提下, 总费用最小的流 (费用为 10), 即最小费用最大流。另一个最大流是从 s 分别运送一个单位到 x 和 y , 但总费用为 11, 不是最优。

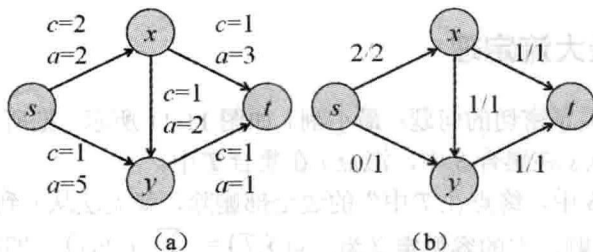


图 11-13 最小费用最大流

在最小费用流问题中, 平行边变得有意义了: 可能会有两条从 u 到 v 的弧, 费用分别为 1 和 2。在没有费用的情况下, 可以把二者合并, 但由于费用的出现, 无法合并这两条弧。再如, 若边 (u, v) 和 (v, u) 均存在, 且费用都是负数, 则“同时从 u 流向 v 和从 v 流向 u ”是个不错的主意。为了更方便地叙述算法, 先假定图中不存在平行边和反向边。这样就可以用两个邻接矩阵 cap 和 cost 保存各边的容量和费用。为了允许反向增广, 规定 $\text{cap}[v][u]=0$ 并且 $\text{cost}[v][u]=-\text{cost}[u][v]$, 表示沿着 (u, v) 的相反方向增广时, 费用减小 $\text{cost}[u][v]$ 。

限于篇幅, 这里直接给出最小费用路算法。和 Edmonds-Karp 算法类似, 但每次用 Bellman-Ford 算法而非 BFS 找增广路。只要初始流是该流量下的最小费用可行流, 每次增广后的新流都是新流量下的最小费用流。另外, 费用值是可正可负的。在下面的代码中, 为了减小溢出的可能, 总费用 cost 采用 long long 来保存。

```
struct Edge {
    int from, to, cap, flow, cost;
    Edge(int u, int v, int c, int f, int w): from(u), to(v), cap(c), flow(f), cost(w)
    {}
};

struct MCMF {
    int n, m;
    vector<Edge> edges;
    vector<int> G[maxn];
    int inq[maxn]; // 是否在队列中
    int d[maxn]; // Bellman-Ford
    int p[maxn]; // 上一条弧
    int a[maxn]; // 可改进量

    void init(int n) {
        this->n = n;
        for(int i = 0; i < n; i++) G[i].clear();
        edges.clear();
    }

    void AddEdge(int from, int to, int cap, int cost) {
        edges.push_back(Edge(from, to, cap, 0, cost));
        edges.push_back(Edge(to, from, 0, 0, -cost));
        m = edges.size();
        G[from].push_back(m-2);
        G[to].push_back(m-1);
    }

    bool BellmanFord(int s, int t, int& flow, long long& cost) {
        for(int i = 0; i < n; i++) d[i] = INF;
```

```

memset(inq, 0, sizeof(inq));
d[s] = 0; inq[s] = 1; p[s] = 0; a[s] = INF;
queue<int> Q;
Q.push(s);
while(!Q.empty()) {
    int u = Q.front(); Q.pop();
    inq[u] = 0;
    for(int i = 0; i < G[u].size(); i++) {
        Edge& e = edges[G[u][i]];
        if(e.cap > e.flow && d[e.to] > d[u] + e.cost) {
            d[e.to] = d[u] + e.cost;
            p[e.to] = G[u][i];
            a[e.to] = min(a[u], e.cap - e.flow);
            if(!inq[e.to]) { Q.push(e.to); inq[e.to] = 1; }
        }
    }
}
if(d[t] == INF) return false;
flow += a[t];
cost += (long long)d[t] * (long long)a[t];
for(int u = t; u != s; u = edges[p[u]].from) {
    edges[p[u]].flow += a[t];
    edges[p[u]^1].flow -= a[t];
}
return true;
}

//需要保证初始网络中没有负权圈
int MincostMaxflow(int s, int t, long long& cost) {
    int flow = 0; cost = 0;
    while(BellmanFord(s, t, flow, cost));
    return flow;
}
};

```

11.4.5 应用举例

首先需要明确一点：虽然本节介绍了最大流的 Edmonds-Karp 算法，但在实践中一般不用这个算法，而是使用效率更高的 Dinic 算法或者 ISAP 算法。这两个算法虽然也不是很难理解，但是较之 Edmonds-Karp 来说还是复杂了许多。另一方面，最小费用流也有更快的算法，但在实践中一般仍用上述算法，因为最小费用流的快速算法（例如网络单纯型法）大

都很复杂，还没有广泛使用。对此，笔者的建议是：理解 Edmonds-Karp 算法的原理（包括正确性证明），但在比赛中使用 Dinic 或者 ISAP。《算法竞赛入门经典——训练指南》对这两个算法有较为详细介绍，还给出了完整的代码。事实上，读者无须搞清楚它们的原理，只需会使用即可。换句话说，可以把它们当作像 STL 一样的黑盒代码。在算法竞赛中，一般把这样的代码称为模板。

二分图匹配。网络流的一个经典的应用是二分图匹配。在图论中，匹配是指两两没有公共点的边集，而二分图是指：可以把结点集分成两部分 X 和 Y ，使得每条边恰好一个端点在 X ，另一个端点在 Y 。换句话说，可以把结点进行二染色（bicoloring），使得同色结点不相邻。为了方便叙述，在画图时一般把 X 结点和 Y 结点画成左右两列。可以证明：一个图是二分图，当且仅当它不含长度为奇数的圈。

常见的二分图匹配问题有两种。第一种是针对无权图的，需要求出包含边数最多的匹配，即二分图的最大基数匹配（maximum cardinality bipartite matching），如图 11-14（a）所示。

这个问题可以这样求解：增加一个源点 s 和一个汇点 t ，从 s 到所有 X 结点各连一条容量为 1 的弧，再从所有 Y 结点各连一条容量为 1 的弧到 t ，最后把每条边变成一条由 X 指向 Y 的有向弧，容量为 1。只要求出 s 到 t 的最大流，则原图中所有流量为 1 的弧对应了最大基数匹配。

第二种是针对带权图的，需要求出边权之和尽量大的匹配，如图 11-14（b）所示。有些题目要求这个匹配本身是完美匹配（perfect matching），即每个点都被匹配到，而有些题目并不对边的数量做出要求，只要权和最大就可以了。下面先考虑前一种情况，即最大权完美匹配（maximum weighted perfect matching）。

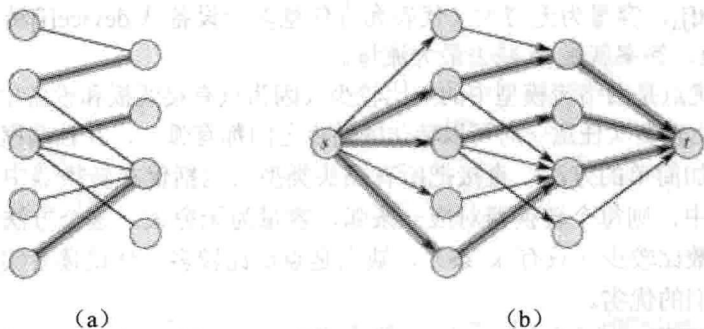


图 11-14 二分图匹配

聪明的读者相信已经找到解决方法了：和最大基数匹配类似，只是原图中所有边的费用为权值的相反数（即前面加一个负号），然后其他边的费用为 0，然后求一个 s 到 t 的最小费用最大流即可。如果从 s 出发的所有弧并不是全部满载（即流量等于容量），则说明完美匹配不存在，问题无解；否则原图中的所有流量为 1 的弧对应最大权完美匹配。

用这样的方法也可以求解第二种情况，即匹配边数没有限制的最大权匹配，只是需要在求解 s - t 最小费用流的过程中记录下流量为 0, 1, 2, 3, ... 时的最小费用流，然后加以比较，细节留给读者思考。

例题 11-7 UNIX 插头 (A Plug for UNIX, UVa753)

有 n 个插座, m 个设备和 k ($n, m, k \leq 100$) 种转换器, 每种转换器都有无限多。已知每个插座的类型, 每个设备的插头类型, 以及每种转换器的插座类型和插头类型。插头和插座类型都用不超过 24 个字母表示, 插头只能插到类型名称相同的插座中。

例如, 有 4 个插座, 类型分别为 A, B, C, D; 有 5 个设备, 插头类型分别为 B, C, B, B, X; 还有 3 种转换器, 分别是 B→X, X→A 和 X→D。这里用 B→X 表示插座类型为 B, 插头类型为 X, 因此一个插头类型为 B 的设备插上这种转换器之后就“变成”了一个插头类型为 X 的设备。转换器可以级联使用, 例如插头类型为 A 的设备依次接上 A→B, B→C, C→D 这 3 个转换器之后会“变成”插头类型为 D 的设备。

要求插的设备尽量多。问最少剩几个不匹配的设备。

【分析】

首先要注意的是: k 个转换器中涉及的插头类型不一定是接线板或者设备中出现过的插头类型。在最坏情况下, 100 个设备, 100 个插座, 100 个转换器最多会出现 400 种插头。当然, 400 种插头的情况肯定是无解的, 但是如果编码不当, 这样的情况可能会让你的程序出现下标越界等运行错误。

笔者第一次尝试本题时使用的方法如下: 转换器有无限多, 所以可以独立计算出每个设备 i 是否可以接上 0 个或多个转换器之后插到第 j 个插座上, 方法是建立有向图 G , 结点表示插头类型, 边表示转换器, 然后使用 Floyd 算法, 计算出任意一种插头类型 a 是否能转化为另一种插头类型 b 。

接下来构造网络: 设设备 i 对应的插头类型编号为 $device[i]$, 插座 i 对应的插头类型编号为 $target[i]$, 则源点 s 到所有 $device[i]$ 连一条弧, 容量为 1, 然后所有 $target[i]$ 到汇点 t 连一条弧, 容量为 1, 对于所有设备 i 和插座 j , 如果 $device[i]$ 可以转化为 $target[j]$, 则从 $device[i]$ 连一条弧到 $target[j]$, 容量为无穷大 (代表允许任意多个设备从 $device[i]$ 转化为 $target[j]$), 最后求 s - t 最大流, 答案就是 m 减去最大流量。

上述算法的优点是网络流模型中的点比较少 (因为只有接线板和设备中出现过的插头类型), 缺点是弧比较多 (任意一对可以转化的结点之间都有弧), 并且编程稍微麻烦一些。

还有一个更加简单的方法: 直接把所有插头类型 (包括仅在转换器中出现的类型) 纳入到网络流模型中, 则每个转换器对应一条弧, 容量为无穷大。这个方法的优点是编程简单, 并且弧的个数比较少 (只有 k 条), 缺点是点数比较多。建议读者实现这两种算法, 然后自行比较它们的优劣。

例题 11-8 矩阵解压 (Matrix Decompressing, UVa 11082)

对于一个 R 行 C 列的正整数矩阵 ($1 \leq R, C \leq 20$), 设 A_i 为前 i 行所有元素之和, B_i 为前 i 列所有元素之和。已知 R, C 和数组 A 和 B , 找一个满足条件的矩阵。矩阵中的元素必须是 1~20 之间的正整数。输入保证有解。

【分析】

首先根据 A_i 和 B_i 计算出第 i 行的元素之和 A'_i 和第 i 列的元素之和 B'_i 。如果把矩阵里的每个数都减 1, 则每个 A'_i 会减少 C , 而每个 B'_i 会减少 R 。这样一来, 每个元素的范围变成了 0~19, 它的好处很快就能看到。

建立一个二分图, 每行对应一个 X 结点, 每列对应一个 Y 结点, 然后增加源点 s 和汇